

W4111 – Introduction to Databases

Lecture 8: Module II (2), NoSQL (2)



Contents

Module II (Cont.)

Module II – DBMS Architecture and Implementation Overview and Reminder

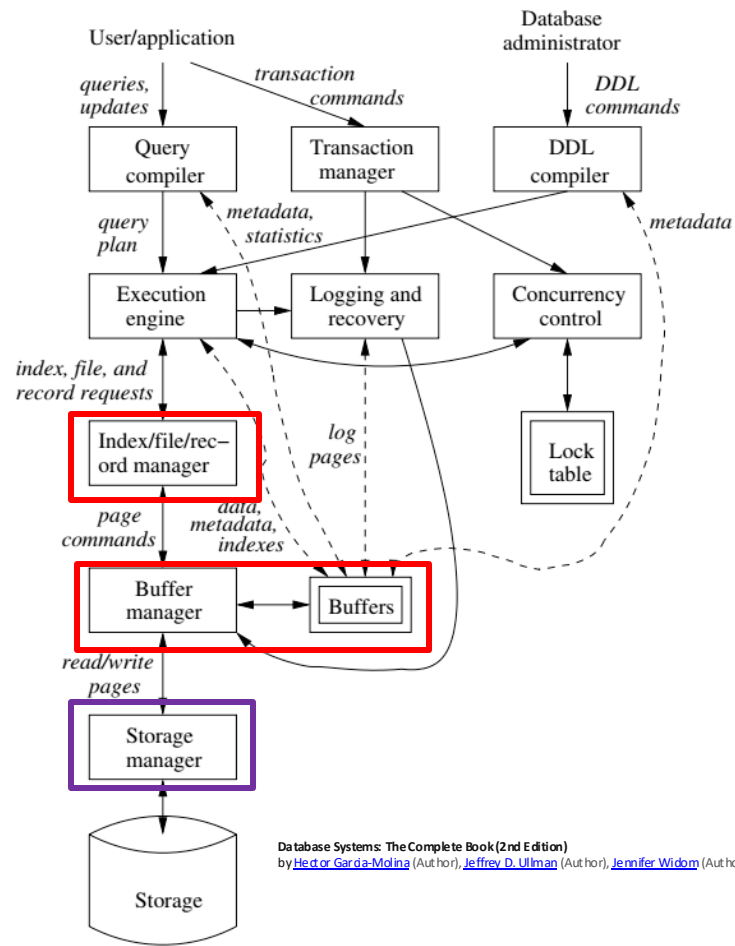
Data Management

Previously

- Load/save things quickly.

Today

- Storage Mgmt. (cont)
- Access data quickly.
- Find things quickly.



Data Storage Structures

(Database Systems Concepts, V7, Ch. 13)

File Organization

- The database is stored as a collection of *files*. Each file is a sequence of *records*. A record is a sequence of fields.
 - One approach
 - Assume record size is fixed
 - Each file has records of one particular type only
 - Different files are used for different relations
- This case is easiest to implement;
We will consider variable length records later
- We assume that records are smaller than a disk block.

From: Database System Concepts, 7th Ed.

Terminology

- A tuple in a relation maps to a *record*. Records may be
 - *Fixed length*
 - *Variable length*
 - *Variable format* (which we will see in Neo4J, DynamoDB, etc).
- A *block*
 - Is the unit of transfer between disks and memory (buffer pools).
 - Contains multiple records, usually but not always from the same relation.
- The *database address space* contains
 - All of the blocks and records that the database manages
 - Including blocks/records containing data
 - And blocks/records containing free space.

Fixed-Length Records

- Simple approach:
 - Store record i starting from byte $n * (i - 1)$, where n is the size of each record.
 - Record access is simple but records may cross blocks
 - Modification: do not allow records to cross block boundaries

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

From: Database System Concepts, 7th Ed.

Fixed-Length Records

- Deletion of record i : alternatives:
 - move records $i + 1, \dots, n$ to $i, \dots, n - 1$
 - move record n to i
 - do not move records, but link all free records on a *free list*

Record 3 deleted

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

From: Database System Concepts, 7th Ed.

Fixed-Length Records

- Deletion of record i : alternatives:
 - move records $i + 1, \dots, n$ to $i, \dots, n - 1$
 - **move record n to i**
 - do not move records, but link all free records on a *free list*

Record 3 deleted and replaced by record 11

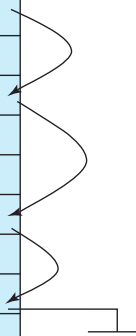
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

From: Database System Concepts, 7th Ed.

Fixed-Length Records

- Deletion of record i : alternatives:
 - move records $i + 1, \dots, n$ to $i, \dots, n - 1$
 - move record n to i
 - **do not move records, but link all free records on a *free list***

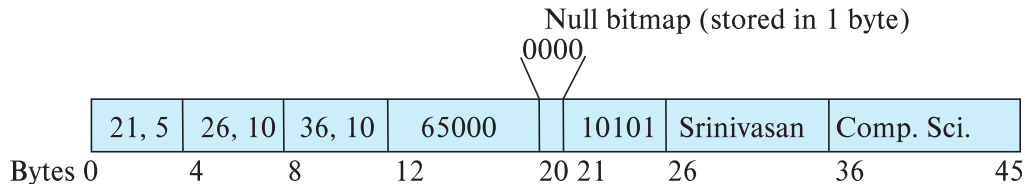
header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



From: Database System Concepts, 7th Ed.

Variable-Length Records

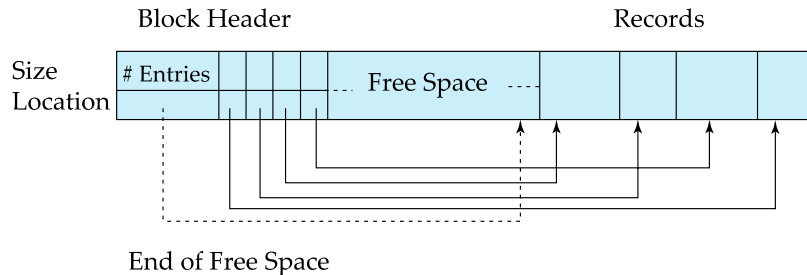
- Variable-length records arise in database systems in several ways:
 - Storage of multiple record types in a file.
 - Record types that allow variable lengths for one or more fields such as strings (**varchar**)
 - Record types that allow repeating fields (used in some older data models).
- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap



From: Database System Concepts, 7th Ed.

Variable-Length Records: Slotted Page Structure

- **Slotted page** header contains:
 - number of record entries
 - end of free space in the block
 - location and size of each record
- Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.
- Pointers should not point directly to record — instead they should point to the entry for the record in header.



From: Database System Concepts, 7th Ed.

Storing Large Objects

- E.g., blob/clob types
- Records must be smaller than pages
- Alternatives:
 - Store as files in file systems
 - Store as files managed by database
 - Break into pieces and store in multiple tuples in separate relation
 - PostgreSQL TOAST

From: Database System Concepts, 7th Ed.

Organization of Records in Files

- **Heap** – record can be placed anywhere in the file where there is space
- **Sequential** – store records in sequential order, based on the value of the search key of each record
- In a **multitable clustering file organization** records of several different relations can be stored in the same file
 - Motivation: store related records on the same block to minimize I/O
- **B⁺-tree file organization**
 - Ordered storage even with inserts/deletes
 - More on this in Chapter 14
- **Hashing** – a hash function computed on search key; the result specifies in which block of the file the record should be placed
 - More on this in Chapter 14

From: Database System Concepts, 7th Ed.

Heap File Organization

- Records can be placed anywhere in the file where there is free space
- Records usually do not move once allocated
- Important to be able to efficiently find free space within file
- **Free-space map**
 - Array with 1 entry per block. Each entry is a few bits to a byte, and records fraction of block that is free
 - In example below, 3 bits per block, value divided by 8 indicates fraction of block that is free

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- Can have second-level free-space map
 - In example below, each entry stores maximum from 4 entries of first-level free-space map
- Free space map written to disk periodically, OK to have wrong (old) values for some entries (will be detected and fixed)

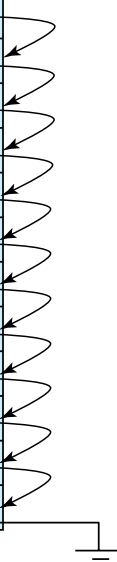
4	7	2	6
---	---	---	---

From: Database System Concepts, 7th Ed.

Sequential File Organization

- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a **search-key**

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

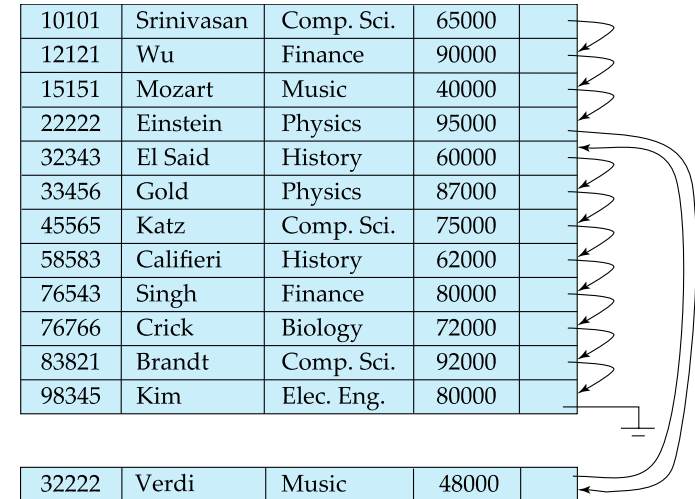


From: Database System Concepts, 7th Ed.

Sequential File Organization (Cont.)

- Deletion – use pointer chains
- Insertion – locate the position where the record is to be inserted
 - if there is free space insert there
 - if no free space, insert the record in an **overflow block**
 - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order

From: Database System Concepts, 7th Ed.



From: Database System Concepts, 7th Ed.

Partitioning

- **Table partitioning:** Records in a relation can be partitioned into smaller relations that are stored separately
- E.g., *transaction* relation may be partitioned into *transaction_2018*, *transaction_2019*, etc.
- Queries written on *transaction* must access records in all partitions
 - Unless query has a selection such as *year=2019*, in which case only one partition is needed
- Partitioning
 - Reduces costs of some operations such as free space management
 - Allows different partitions to be stored on different storage devices
 - E.g., *transaction* partition for current year on SSD, for older years on magnetic disk

From: Database System Concepts, 7th Ed.

Column-Oriented Storage

- Also known as **columnar representation**
- Store each attribute of a relation separately
- Example

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

record 0
record 1
record 2
record 11
record 4
record 5
record 6
record 7
record 8
record 9
record 10

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
98345	Kim	Elec. Eng.	80000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000

From: Database System Concepts, 7th Ed.

Columnar Representation

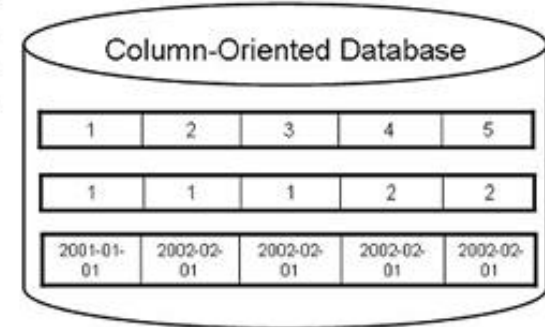
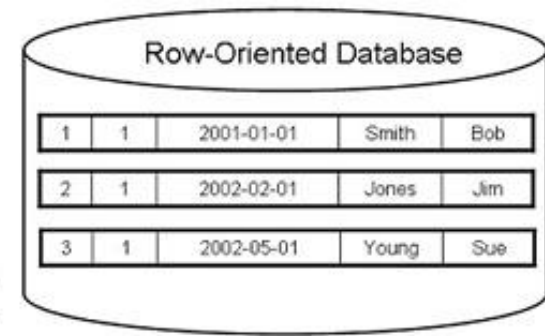
- Benefits:
 - Reduced IO if only some attributes are accessed
 - Improved CPU cache performance
 - Improved compression
 - **Vector processing** on modern CPU architectures
- Drawbacks
 - Cost of tuple reconstruction from columnar representation
 - Cost of tuple deletion and update
 - Cost of decompression
- Columnar representation found to be more efficient for decision support than row-oriented representation
- Traditional row-oriented representation preferable for transaction processing
- Some databases support both representations
 - Called **hybrid row/column stores**

From: Database System Concepts, 7th Ed.

Row vs Column

- Columnar and Row are both
 - Relational
 - Support SQL operations
- But differ in data storage
 - Row keeps row data together in blocks.
 - Columnar keys column data together in blocks.
- This determines performance for different types of query, e.g.
 - Columnar is extremely powerful for BI systems
 - Aggregation ops, e.g. SUM, AVG
 - PROJECT (do not load all of the row) to get
 - Row is powerful for OLTP. Transaction typically create and retrieve
 - One row at a time
 - All the columns of a single row.

Emp_no	Dept_id	Hire_date	Emp_ln	Emp_fn
1	1	2001-01-01	Smith	Bob
2	1	2002-02-01	Jones	Jim
3	1	2002-05-01	Young	Sue
4	2	2003-02-01	Stemle	Bill
5	2	1999-06-15	Aurora	Jack
6	3	2000-08-15	Jung	Laura



Columnar File Representation

<https://towardsdatascience.com/demystifying-the-parquet-file-format-13adb0206705>

Row-Based Storage Layout

String	Int	Date
a	1	2020-01
b	2	2020-02
c	3	2020-03



Column-Based Storage Layout

String	Int	Date
a	1	2020-01
b	2	2020-02
c	3	2020-03



Hybrid-Based Storage Layout (row group size = 2)

String	Int	Date
a	1	2020-01
b	2	2020-02
c	3	2020-03



Final group only has 1 row

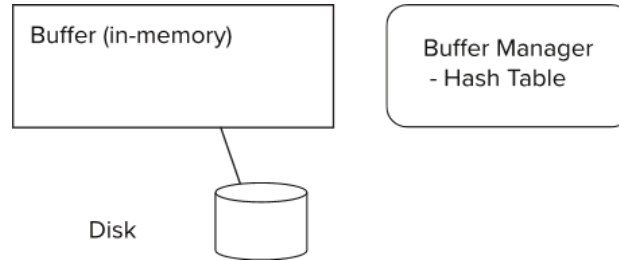
Apache parquet is an open-source file format that provides efficient storage and fast read speed. It uses a hybrid storage format which sequentially stores chunks of columns, lending to high performance when selecting and filtering data. On top of strong compression algorithm support ([snappy](#), [gzip](#), [LZO](#)), it also provides some clever tricks for reducing file scans and encoding repeat variables.

Buffer Pool Management



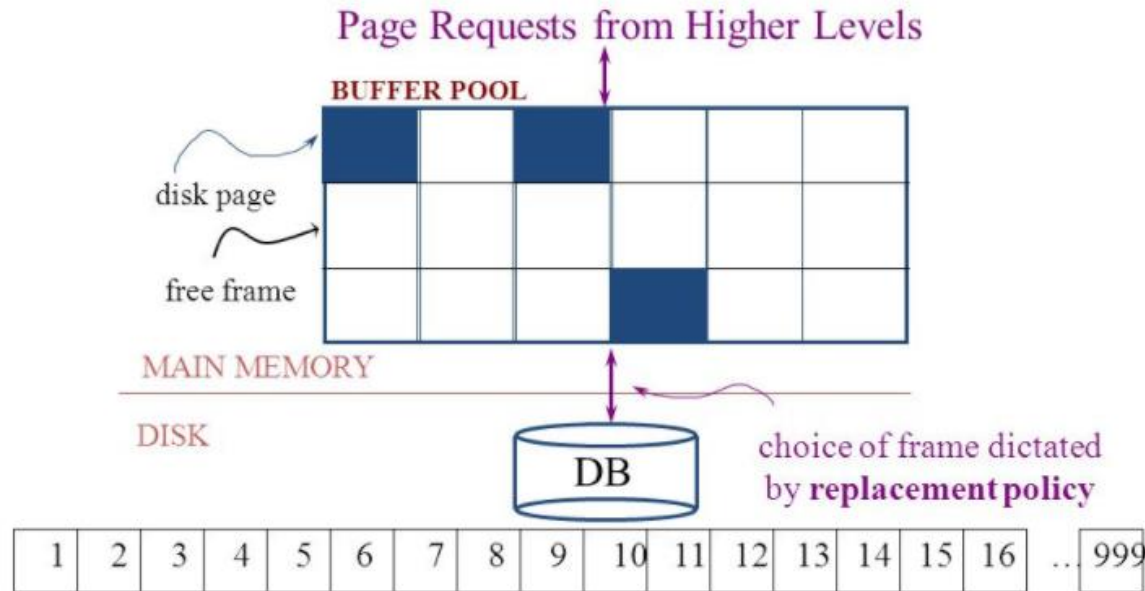
Storage Access

- Blocks are units of both storage allocation and data transfer.
- Database system seeks to minimize the number of block transfers between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.
- **Buffer** – portion of main memory available to store copies of disk blocks.
- **Buffer manager** – subsystem responsible for allocating buffer space in main memory.



The Logical Concept

- The DBMS and queries can only manipulate in-memory blocks and records.
- A very, very, very small fraction of all blocks fit in memory.





Buffer Manager

- Programs call on the buffer manager when they need a block from disk.
 - If the block is already in the buffer, buffer manager returns the address of the block in main memory
 - If the block is not in the buffer, the buffer manager
 - Allocates space in the buffer for the block
 - Replacing (throwing out) some other block, if required, to make space for the new block.
 - Replaced block written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
 - Reads the block from the disk to the buffer, and returns the address of the block in main memory to requester.



Buffer Manager

- **Buffer replacement strategy** (details coming up!)
- **Pinned block:** memory block that is not allowed to be written back to disk
 - **Pin** done before reading/writing data from a block
 - **Unpin** done when read /write is complete
 - Multiple concurrent pin/unpin operations possible
 - Keep a pin count, buffer block can be evicted only if pin count = 0
- **Shared and exclusive locks on buffer**
 - Needed to prevent concurrent operations from reading page contents as they are moved/reorganized, and to ensure only one move/reorganize at a time
 - Readers get shared lock, updates to a block require exclusive lock
 - **Locking rules:**
 - Only one process can get exclusive lock at a time
 - Shared lock cannot be concurrently with exclusive lock
 - Multiple processes may be given shared lock concurrently



Buffer-Replacement Policies

- Most operating systems replace the block **least recently used** (LRU strategy)
 - Idea behind LRU – use past pattern of block references as a predictor of future references
 - LRU can be bad for some queries
- Queries have well-defined access patterns (such as sequential scans), and a database system can use the information in a user's query to predict future references
- Mixed strategy with hints on replacement strategy provided by the query optimizer is preferable
- Example of bad access pattern for LRU: when computing the join of 2 relations r and s by a nested loops
 - for each tuple tr of r do
 - for each tuple ts of s do
 - if the tuples tr and ts match ...



Buffer-Replacement Policies (Cont.)

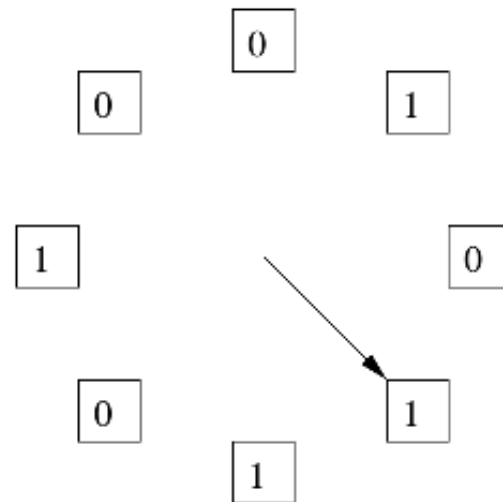
- **Toss-immediate** strategy – frees the space occupied by a block as soon as the final tuple of that block has been processed
- **Most recently used (MRU) strategy** – system must pin the block currently being processed. After the final tuple of that block has been processed, the block is unpinning, and it becomes the most recently used block.
- Buffer manager can use statistical information regarding the probability that a request will reference a particular relation
 - E.g., the data dictionary is frequently accessed. Heuristic: keep data-dictionary blocks in main memory buffer
- Operating system or buffer manager may reorder writes
 - Can lead to corruption of data structures on disk
 - E.g., linked list of blocks with missing block on disk
 - File systems perform consistency check to detect such situations
 - Careful ordering of writes can avoid many such problems

Replacement Policy

- The *replacement policy* is one of the most important factors in database management system implementation and configuration.
- A very simple, introductory explanation is (https://en.wikipedia.org/wiki/Cache_replacement_policies).
 - There are a lot of possible policies.
 - The *most* efficient caching algorithm would be to always discard the information that will not be needed for the longest time in the future. This optimal result is referred to as [Bélády's](#) optimal algorithm/simply optimal replacement policy or [the clairvoyant algorithm](#).
- All implementable policies are an attempt to approximate knowledge of the future based on knowledge of the past.
- Least Recently Used is based on the simplest assumption
 - The information that will not be needed for the longest time.
 - Is the information that has not been accessed for the longest time.

The “Clock Algorithm”

- LRU is (perceived to be) expensive
 - Maintain timestamp for each block.
 - Update and resort blocks on access.
- The “Clock Algorithm” is a less expensive approximation.
 - Arrange the frames (places blocks can go) into a logical circle like
 - Each frame is marked 0 or 1.
 - Set to 1 when block added to frame.
 - Or when application accesses a block in frame.
 - Replacement choice
 - Sweep second hand clockwise one frame at a time.
 - If bit is 0, choose for replacement.
 - If bit is 1, set bit to zero and go to next frame.
- The basic idea is. On a clock face
 - If the second hand is currently at 27 seconds.
 - The 28 second tick mark is “the least recently touched mark.”



Replacement Algorithm

The algorithms are more sophisticated in the real world, e.g.

- “Scans” are common, e.g. go through a large query result in order (will be more clear when discussing cursors).
 - The engine knows the current position in the result set.
 - Uses the sort order to determine which records will be accessed soon.
 - Tags those blocks as not replaceable.
 - (A form of clairvoyance).
- Not all users/applications are equally “important.”
 - Classify users/applications into priority 1, 2 and 3.
 - Sub-allocate the buffer pool into pools P1, P2 and P3.
 - Apply LRU within pools and adjust pool sizes based on relative importance.
 - This prevents
 - A high access rate, low-priority application from taking up a lot of frames
 - Result in low access, high priority applications not getting buffer hits.



Optimization of Disk Block Access (Cont.)

- Buffer managers support **forced output** of blocks for the purpose of recovery (more in Chapter 19)
- **Nonvolatile write buffers** speed up disk writes by writing blocks to a non-volatile RAM or flash buffer immediately
 - *Writes can be reordered to minimize disk arm movement*
- **Log disk** – a disk devoted to writing a sequential log of block updates
 - Used exactly like nonvolatile RAM
 - Write to log disk is very fast since no seeks are required
- **Journaling file systems** write data in-order to NV-RAM or log disk
 - Reordering without journaling: risk of corruption of file system data

Indexes



Basic Concepts

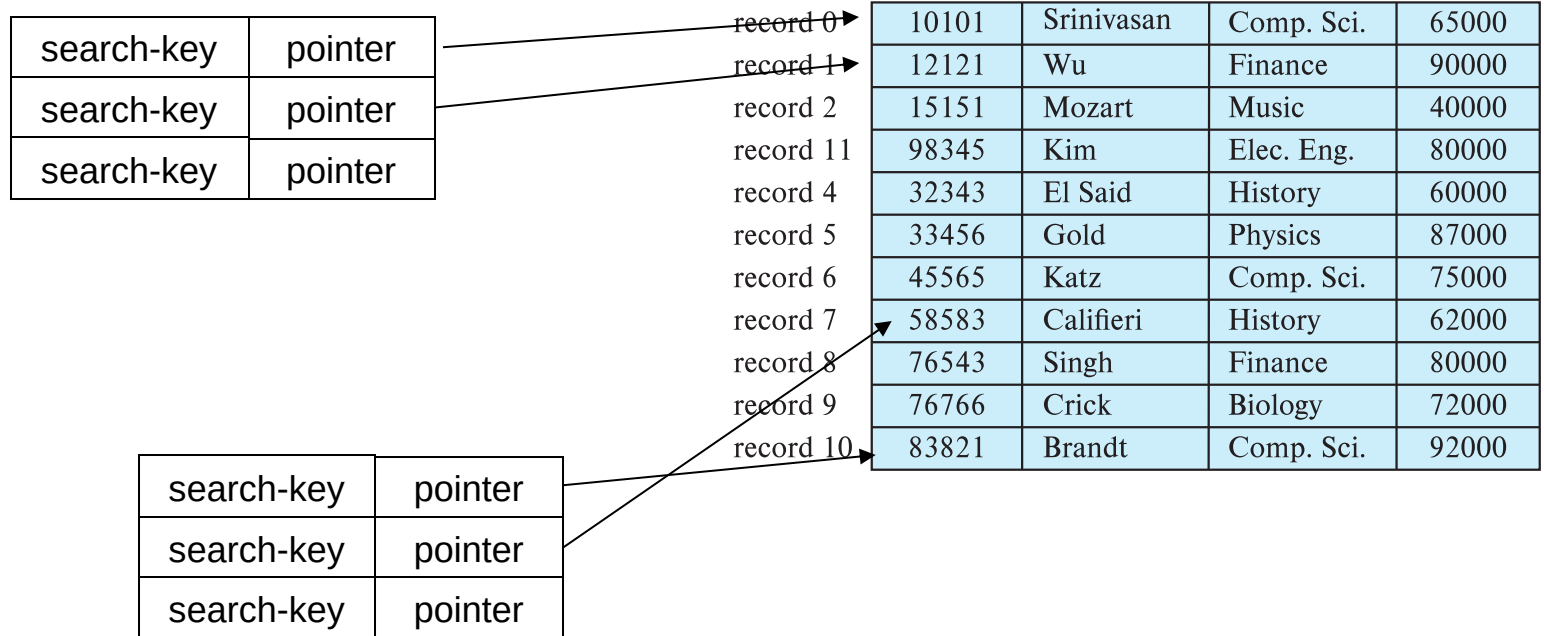
- Indexing mechanisms used to speed up access to desired data.
 - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer
------------	---------

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
 - **Ordered indices:** search keys are stored in sorted order
 - **Hash indices:** search keys are distributed uniformly across “buckets” using a “hash function”.

(“c:\”, LBN, offset)

search-key	pointer
search-key	pointer
search-key	pointer





Ordered Indices

- In an **ordered index**, index entries are stored sorted on the search key value.
- **Clustering index**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
 - Also called **primary index**
 - The search key of a primary index is usually but not necessarily the primary key.
- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file. Also called **nonclustering index**.
- **Index-sequential file**: sequential file ordered on a search key, with a clustering index on the search key.



Dense Index Files

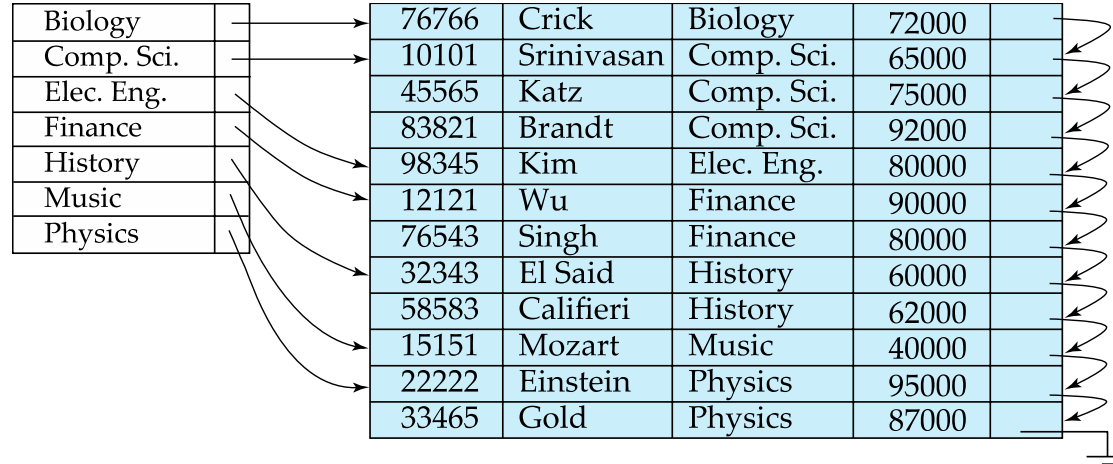
- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation

10101	→	10101	Srinivasan	Comp. Sci.	65000	↙
12121	→	12121	Wu	Finance	90000	↙
15151	→	15151	Mozart	Music	40000	↙
22222	→	22222	Einstein	Physics	95000	↙
32343	→	32343	El Said	History	60000	↙
33456	→	33456	Gold	Physics	87000	↙
45565	→	45565	Katz	Comp. Sci.	75000	↙
58583	→	58583	Califieri	History	62000	↙
76543	→	76543	Singh	Finance	80000	↙
76766	→	76766	Crick	Biology	72000	↙
83821	→	83821	Brandt	Comp. Sci.	92000	↙
98345	→	98345	Kim	Elec. Eng.	80000	↙



Dense Index Files (Cont.)

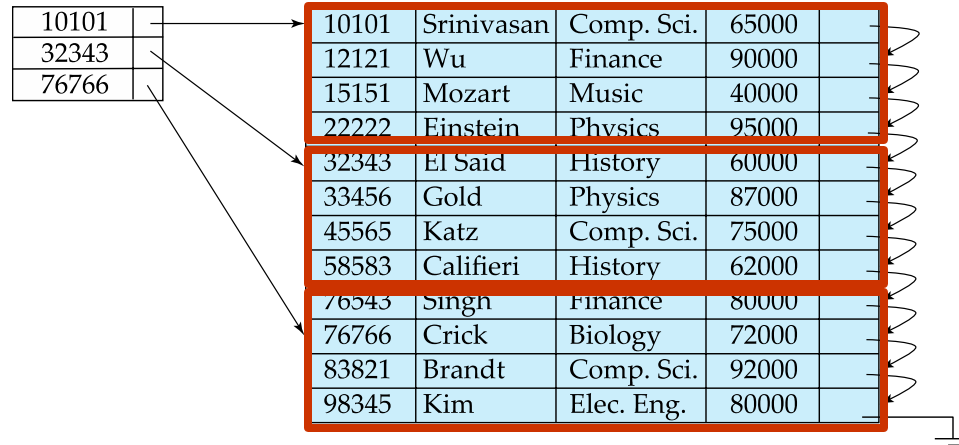
- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*





Sparse Index Files

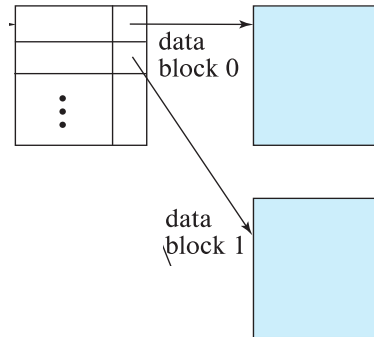
- **Sparse Index:** contains index records for only some search-key values.
 - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
 - Find index record with largest search-key value $< K$
 - Search file sequentially starting at the record to which the index record points





Sparse Index Files (Cont.)

- Compared to dense indices:
 - Less space and less maintenance overhead for insertions and deletions.
 - Generally slower than dense index for locating records.
- **Good tradeoff:**
 - for clustered index: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.

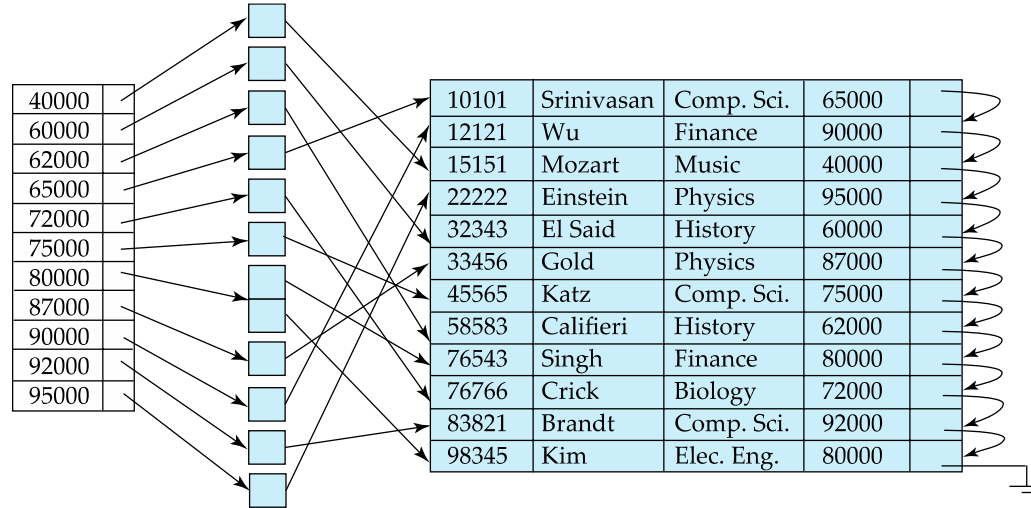


- For unclustered index: sparse index on top of dense index (multilevel index)



Secondary Indices Example

- Secondary index on salary field of instructor



- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense



Indices on Multiple Keys

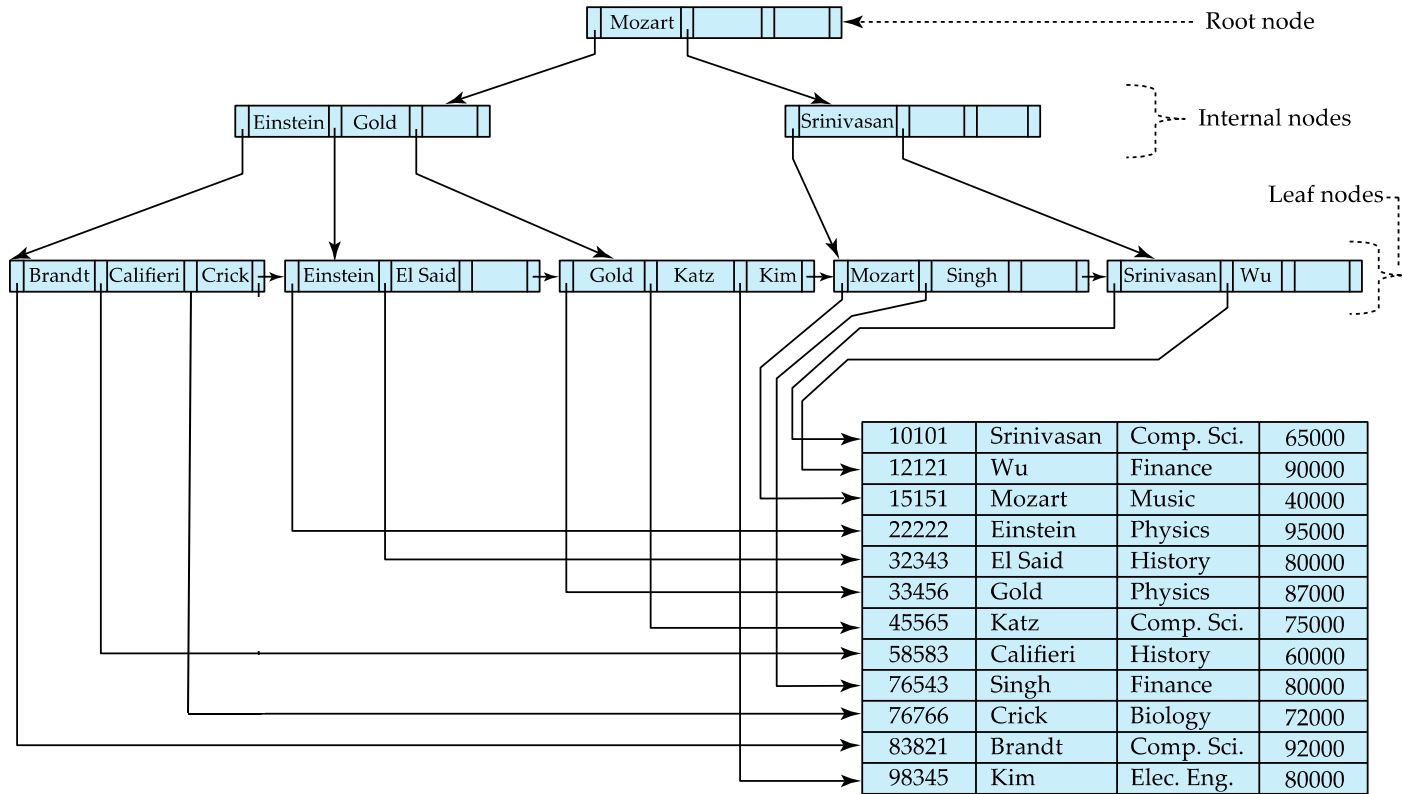
- **Composite search key**
 - E.g., index on *instructor* relation on attributes (*name*, *ID*)
 - Values are sorted lexicographically
 - E.g. (John, 12121) < (John, 13514) and (John, 13514) < (Peter, 11223)
 - Can query on just *name*, or on (*name*, *ID*)

- (nameLast, nameFirst, birthyear)
 - nameLast [nameLast = "Ferguson"] [nameLast like "Fer%"]
 - nameLast, nameFirst
 - nameLast, nameFirst, birthyear

- NOT and index on
 - nameFirst, nameLast
 - birthyear
 - nameLast like [%er%]



Example of B⁺-Tree





B⁺-Tree Index Files (Cont.)

A B⁺-tree is a rooted tree satisfying the following properties:

- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.



B⁺-Tree Node Structure

- Typical node

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

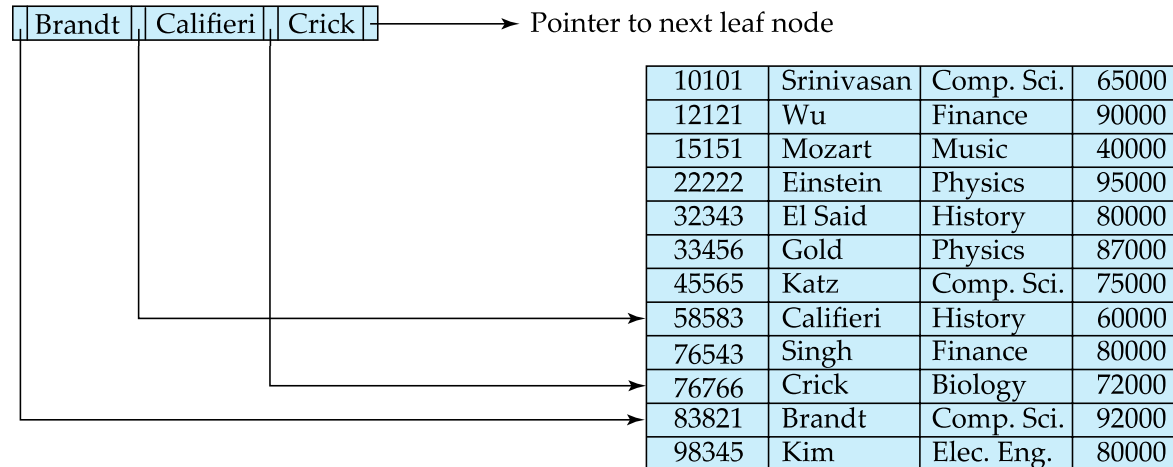
- K_i are the search-key values
 - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$

(Initially assume no duplicate keys, address duplicates later)



- For $i = 1, 2, \dots, n-1$, pointer P_i points to a file record with search-key value K_i ,
- If L_i, L_j are leaf nodes and $i < j$, L_i 's search-key values are less than or equal to L_j 's search-key values
- P_n points to next leaf node in search-key order





Non-Leaf Nodes in B⁺-Trees

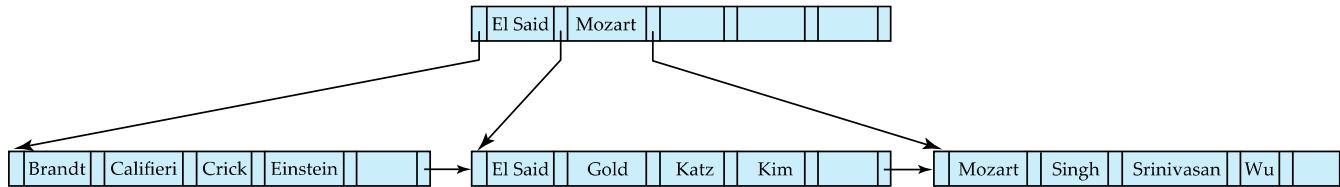
- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with m pointers:
 - All the search-keys in the subtree to which P_1 points are less than K_1
 - For $2 \leq i \leq n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
 - All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}
 - General structure

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------



Example of B⁺-tree

- B⁺-tree for *instructor* file ($n = 6$)



- Leaf nodes must have between 3 and 5 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$).
- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil n/2 \rceil$ and n with $n = 6$).
- Root must have at least 2 children.



Observations about B⁺-trees

- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B⁺-tree form a hierarchy of sparse indices.
- The B⁺-tree contains a relatively small number of levels
 - Level below root has at least $2 * \lceil n/2 \rceil$ values
 - Next level has at least $2 * \lceil n/2 \rceil * \lceil n/2 \rceil$ values
 - .. etc.
 - If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
 - thus searches can be conducted efficiently.
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time (as we shall see).

Show the Simulator

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>



Hashing



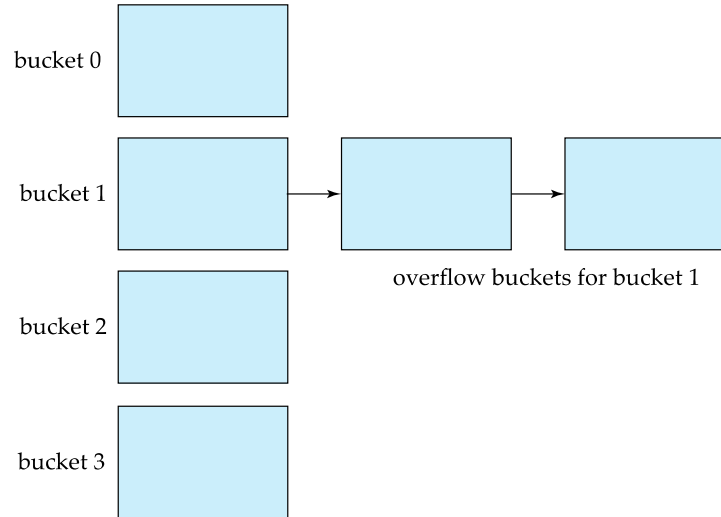
Static Hashing

- A **bucket** is a unit of storage containing one or more entries (a bucket is typically a disk block).
 - we obtain the bucket of an entry from its search-key value using a **hash function**
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B .
- Hash function is used to locate entries for access, insertion as well as deletion.
- Entries with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate an entry.
- In a **hash index**, buckets store entries with pointers to records
- In a **hash file-organization** buckets store records



Handling of Bucket Overflows (Cont.)

- **Overflow chaining** – the overflow buckets of a given bucket are chained together in a linked list.
- Above scheme is called **closed addressing** (also called **closed hashing** or **open hashing** depending on the book you use)
 - An alternative, called **open addressing** (also called **open hashing** or **closed hashing** depending on the book you use) which does not use overflow buckets, is not suitable for database applications.





Example of Hash File Organization

Hash file organization of *instructor* file, using *dept_name* as key.

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7



Deficiencies of Static Hashing

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time.
 - If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows.
 - If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
 - If database shrinks, again space will be wasted.
- One solution: periodic re-organization of the file with a new hash function
 - Expensive, disrupts normal operations
- Better solution: allow the number of buckets to be modified dynamically.

Show the Simulator

<http://iswsa.acm.org/mphf/openDSAPerfectHashAnimation/perfectHashAV.html>
<https://opensa-server.cs.vt.edu/ODSA/AV/Development/hashAV.html>

NoSQL

Concepts

Simplistic Classification (<https://medium.com/swlh/4-types-of-NoSQL-databases-d88ad21f7d3b>)

Relational is the foundational model.

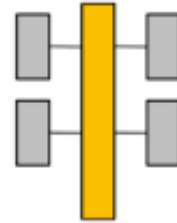
We covered graphs and examples.

SQL Database

Relational



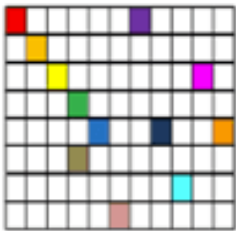
Analytical (OLAP)



We will see OLAP in a future lecture.

NoSQL Database

Column-Family



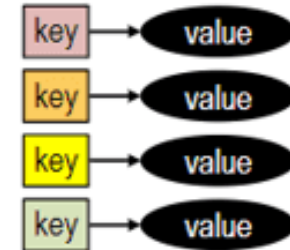
Graph



Document



Key-Value

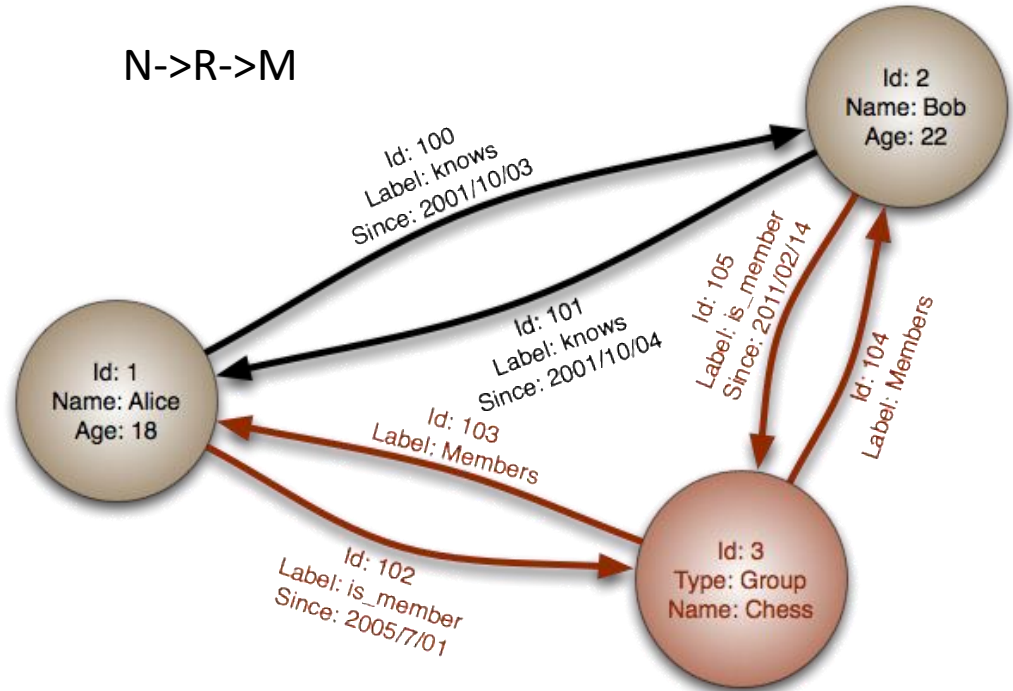


Subject of this lecture and part of HW3

GraphDB

Graph Database

- Exactly what it sounds like
- Two core types
 - Node
 - Edge (link)
- Nodes and Edges have
 - Label(s) = “Kind”
 - Properties (free form)
- Query is of the form
 - $p1(n)-p2(e)-p3(m)$
 - n, m are nodes; e is an edge
 - $p1, p2, p3$ are predicates on labels

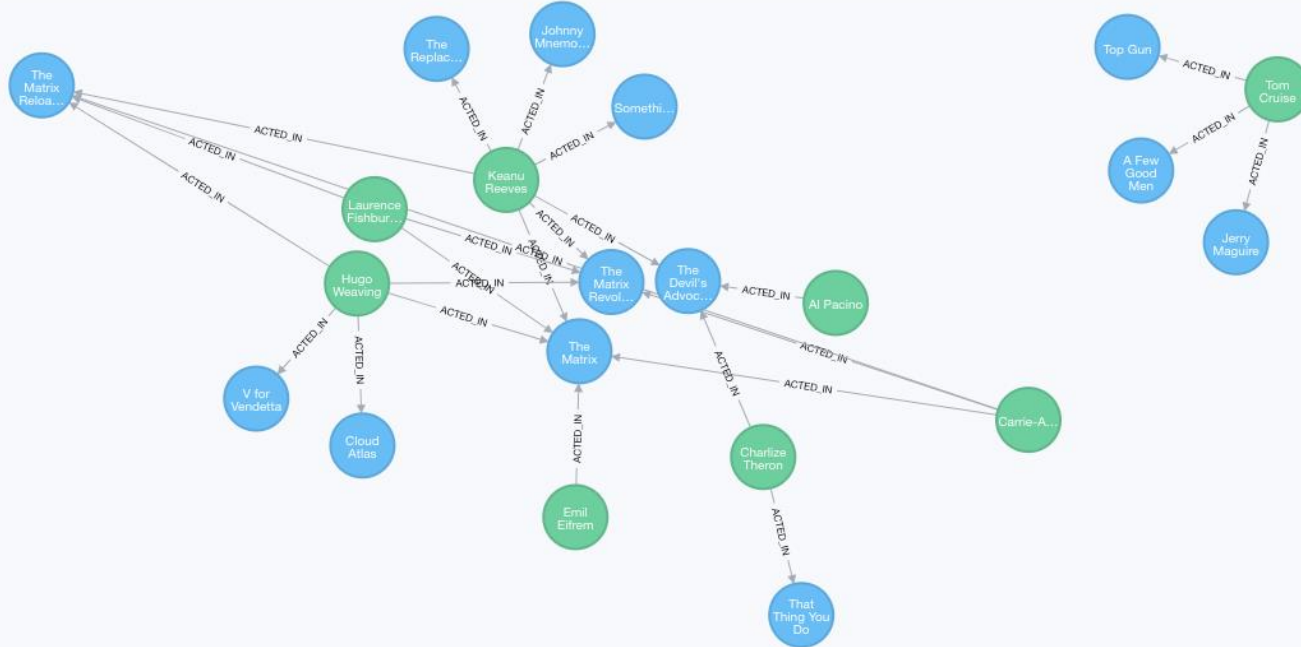


Neo4J Graph Query

```
$ MATCH p=()-[r:ACTED_IN]->() RETURN p LIMIT 25
```

*(21) **Movie(13)** **Person(8)**

*(25) **ACTED_IN(25)**



Displaying 21 nodes, 25 relationships.

AUTO-COMPLETE ☒

Why Graph Databases?

- Schema Less and Efficient storage of Semi Structured Information
- No O/R mismatch – very natural to map a graph to an Object Oriented language like Ruby.
- Express Queries as Traversals. Fast deep traversal instead of slow SQL queries that span many table joins.
- Very natural to express graph related problem with traversals (recommendation engine, find shortest path etc..)
- Seamless integration with various existing programming languages.
- ACID Transaction with rollbacks support.
- Whiteboard friendly – you use the language of node, properties and relationship to describe your domain (instead of e.g. UML) and there is no need to have a complicated O/R mapping tool to implement it in your database. You can say that Neo4j is “Whiteboard friendly” !(<http://video.neo4j.org/JHU6F/live-graph-session-how-allison-knows-james/>)

Graph Databases

Graph databases are

- Extremely fast for some queries and data models.
- Implement a language that vastly simplifies writing queries.

Social Network “path exists” Performance

- Experiment:
 - ~1k persons
 - Average 50 friends per person
 - `pathExists(a,b)`
limited to depth 4

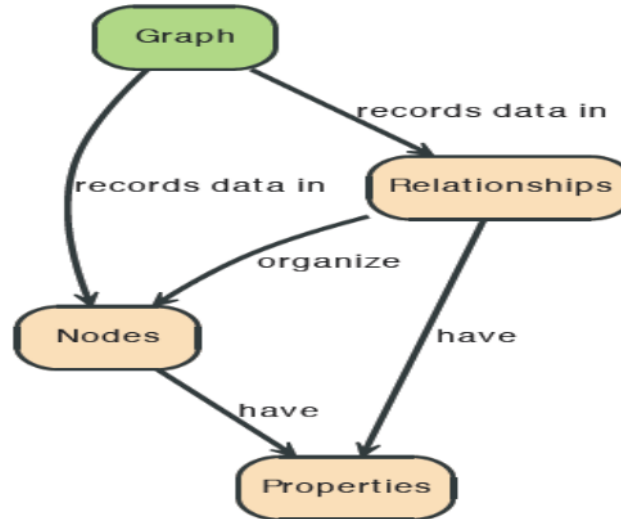
	# persons	query time
Relational database	1000	2000ms
Neo4j	1000	2ms
Neo4j	1000000	2ms

What are graphs good for?

- Recommendations
- Business intelligence
- Social computing
- Geospatial
- Systems management
- Web of things
- Genealogy
- Time series data
- Product catalogue
- Web analytics
- Scientific computing (especially bioinformatics)
- Indexing your *slow* RDBMS
- And much more!

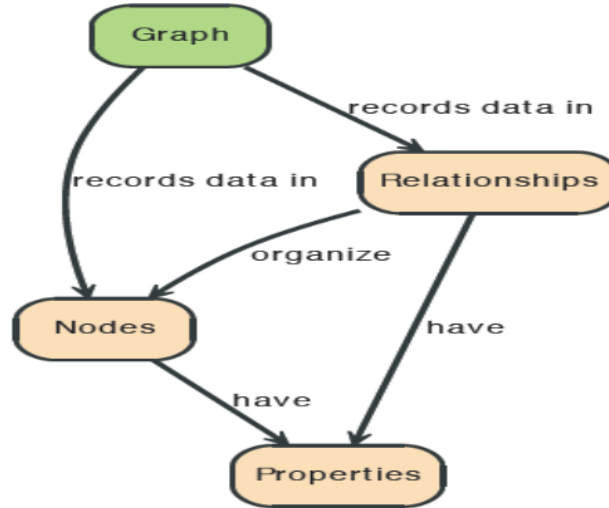
Graphs

- “A Graph —records data in→ Nodes —which have→ Properties”



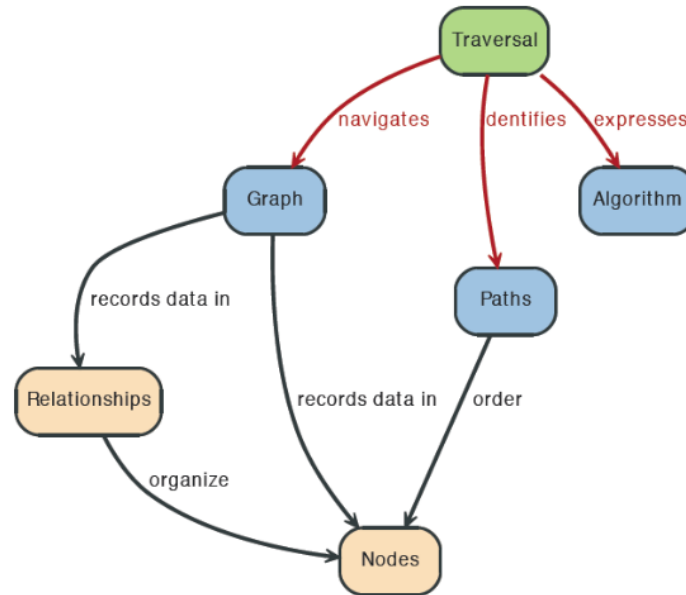
Graphs

- “Nodes —are organized by→ Relationships — which also have→ Properties”



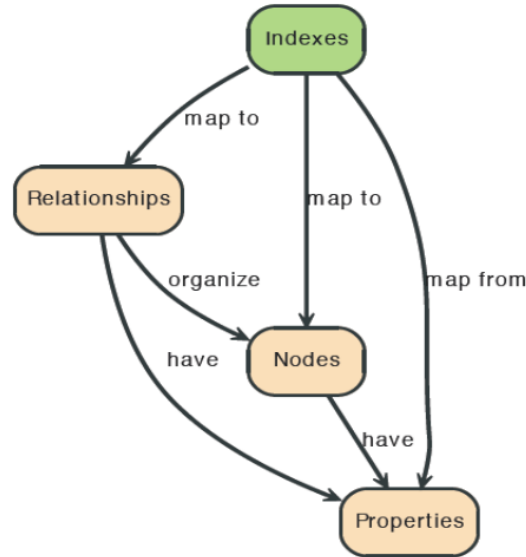
Query a graph with Traversal

- “A Traversal —navigates→ a Graph; it —identifies→ Paths —which order→ Nodes”

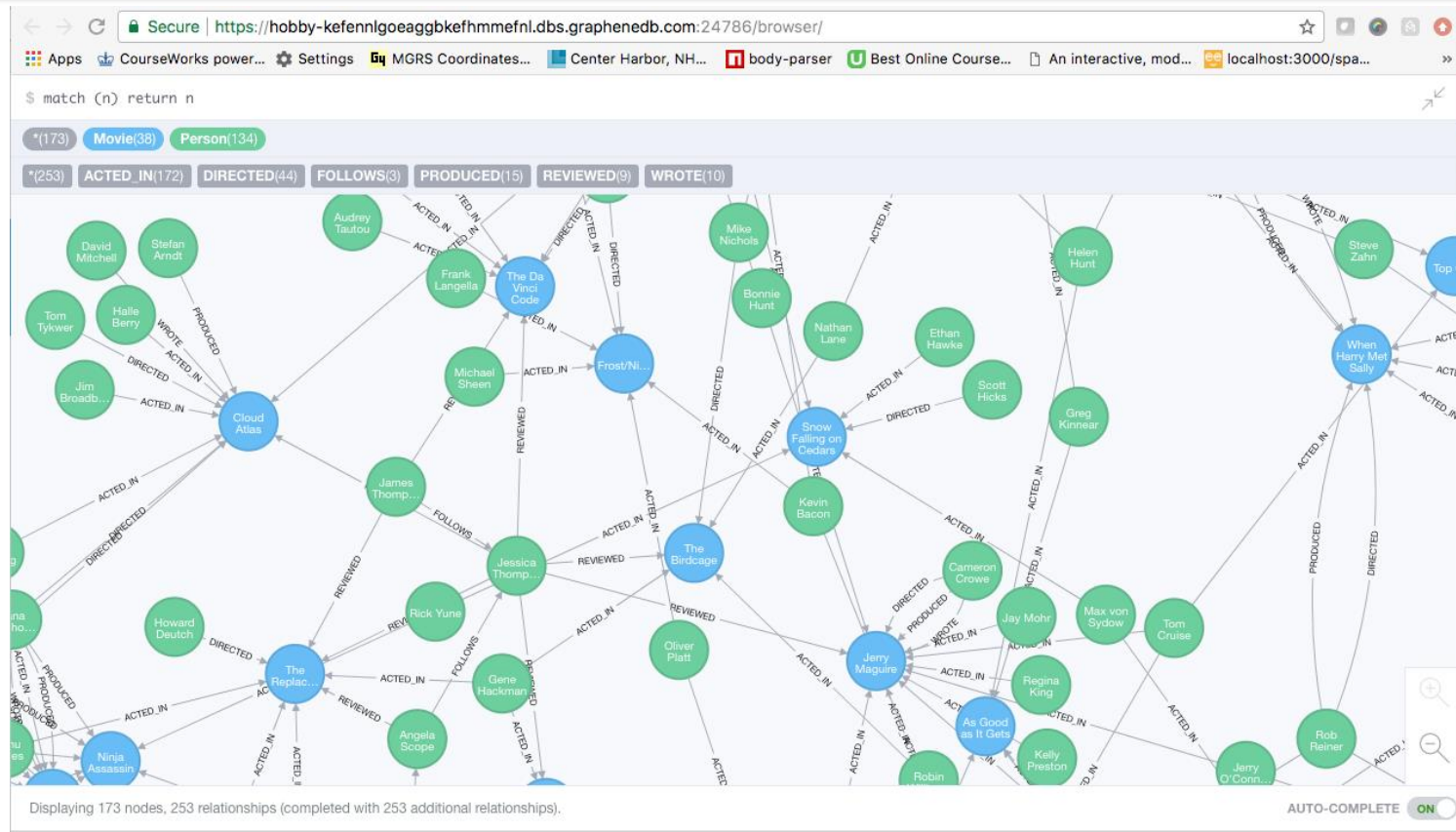


Indexes

- “An Index —maps from→ Properties —to either→ Nodes or Relationships”



A Graph Database (Sample)



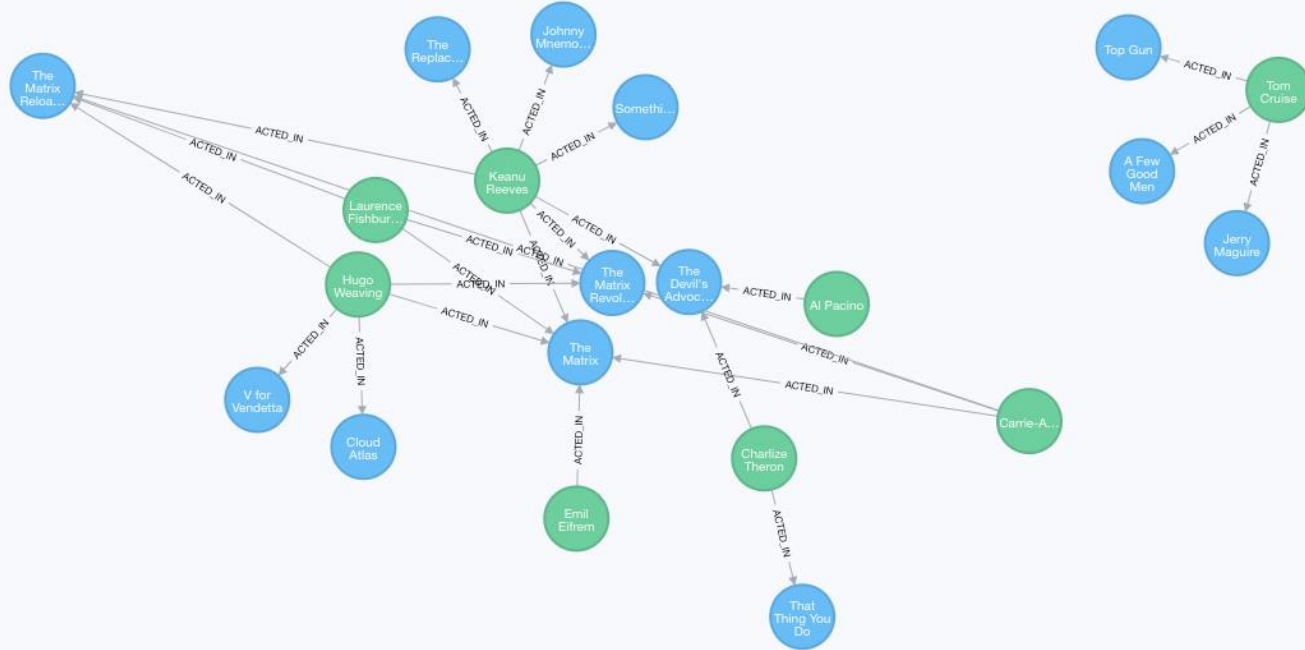
Neo4J Graph Query

Who acted in which movies?

```
$ MATCH p=()-[r:ACTED_IN]->() RETURN p LIMIT 25
```

*(21) Movie(13) Person(8)

*(25) ACTED_IN(25)



Displaying 21 nodes, 25 relationships.

AUTO-COMPLETE ☒

Big Deal. That is just a JOIN.

- Yup. But that is simple.
- Try writing the queries below in SQL.

The Movie Graph Recommend

Let's recommend new co-actors for Tom Hanks. A basic recommendation approach is to find connections past an immediate neighborhood which are themselves well connected.

For Tom Hanks, that means:

1. Find actors that Tom Hanks hasn't yet worked with, but his co-actors have.
2. Find someone who can introduce Tom to his potential co-actor.

Extend Tom Hanks co-actors, to find co-co-actors who haven't work with Tom Hanks...

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
WHERE NOT (tom)-[:ACTED_IN]->(m2)
RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC
```

Find someone to introduce Tom Hanks to Tom Cruise

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
RETURN tom, m, coActors, m2, cruise
```

Recommend

```
1 MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
2      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
3 WHERE NOT (tom)-[:ACTED_IN]->(m2)
4 RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC
```



\$ MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors), (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors) ...



	Recommended	Strength
Rows	Tom Cruise	5
A	Zach Grenier	5
Text	Helen Hunt	4
</>	Cuba Gooding Jr.	4
Code	Keanu Reeves	4
	Tom Skerritt	3
	Carrie-Anne Moss	3
	Val Kilmer	3
	Bruno Kirby	3
	Philip Seymour Hoffman	3
	Billy Crystal	3
	Carrie Fisher	3

```

1 MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-(coActors),
2     (coActors)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
3 RETURN tom, m, coActors, m2, cruise

```

```
$ MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-(coActors), (coActors)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
```

Graph

*(13) Movie(8) Person(5)

*(16) ACTED_IN(16)

Rows

Text

Code



Which actors have worked with both Tom Hanks and Tom Cruise?

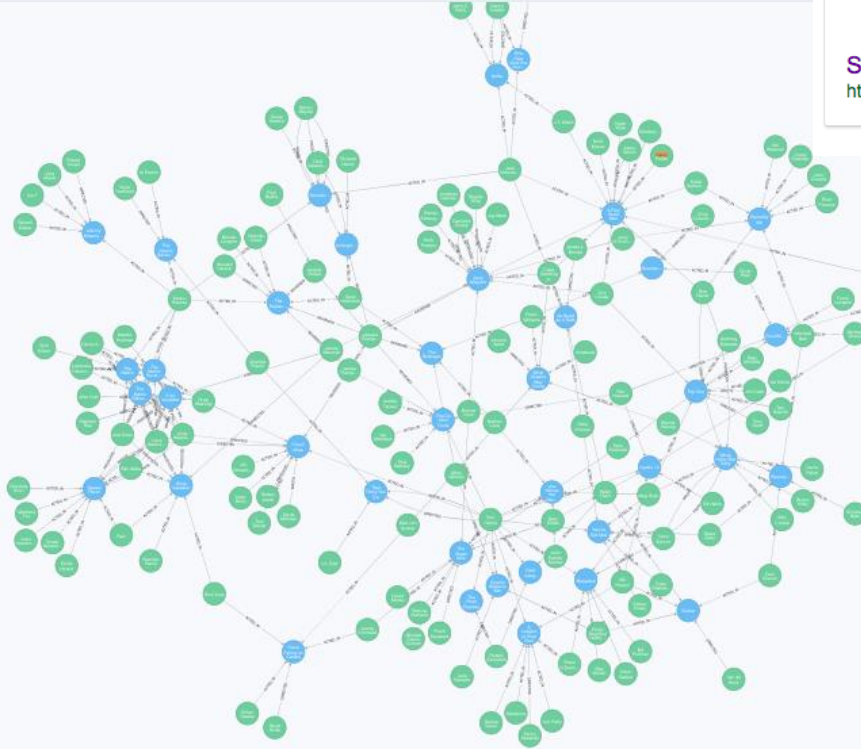
Displaying 13 nodes, 16 relationships (completed with 16 additional relationships).

AUTO-COMPLETE ☒

```
$ MATCH (s:Person { name: 'Kevin Bacon' })-[*0..6]-(m) return s,m
```

*(171) **Movie(38)** **Person(133)**

*(253) **ACTED_IN(172)** **DIRECTED(44)** **FOLLOWS(3)** **PRODUCED(15)** **REVIEWED(9)** **WROTE(10)**



Six Degrees of Kevin Bacon is a parlour game based on the "six degrees of separation" concept, which posits that any two people on Earth are six or fewer acquaintance links apart. Movie buffs challenge each other to find the shortest path between an arbitrary actor and prolific actor **Kevin Bacon**.



[Six Degrees of Kevin Bacon - Wikipedia](https://en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon)

https://en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon

[About this result](#)

[Feedback](#)

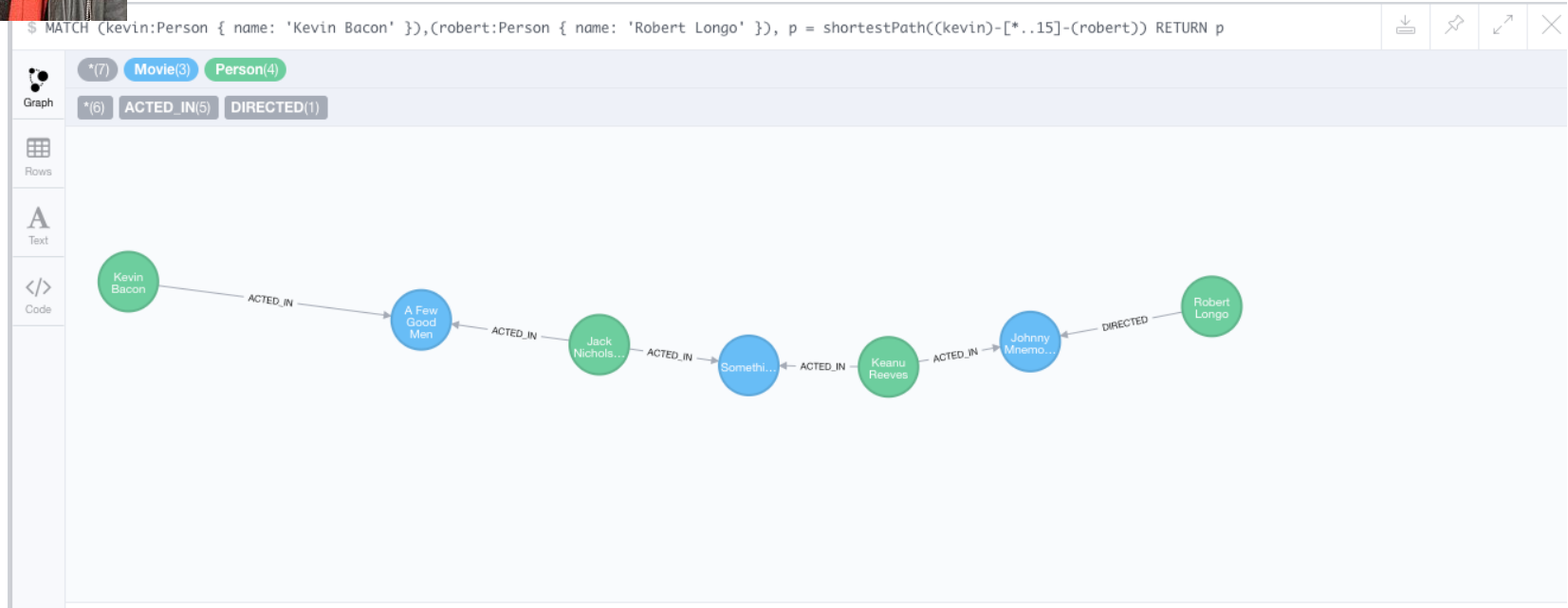
Six Degrees of Kevin Bacon

Game



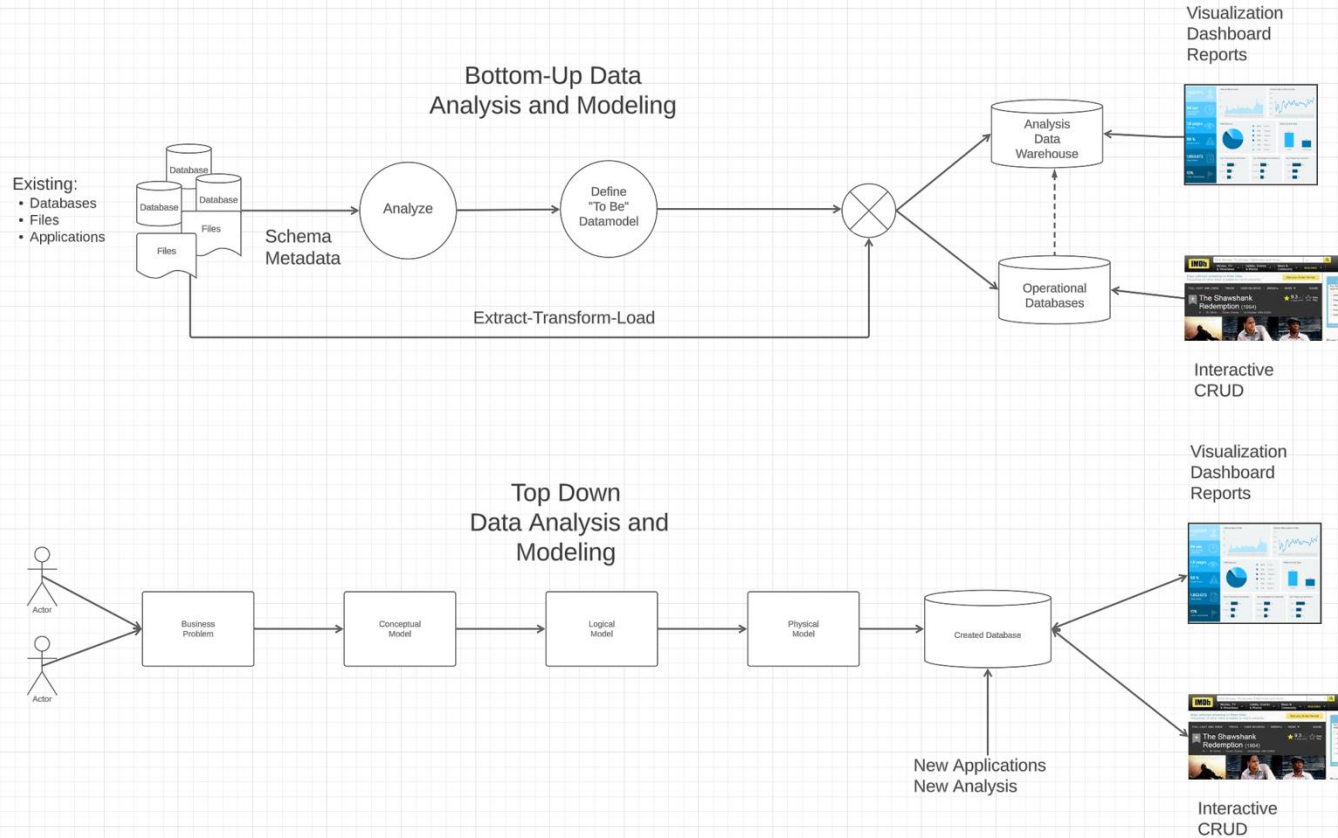


How do you get from Kevin Bacon to Robert Longo?

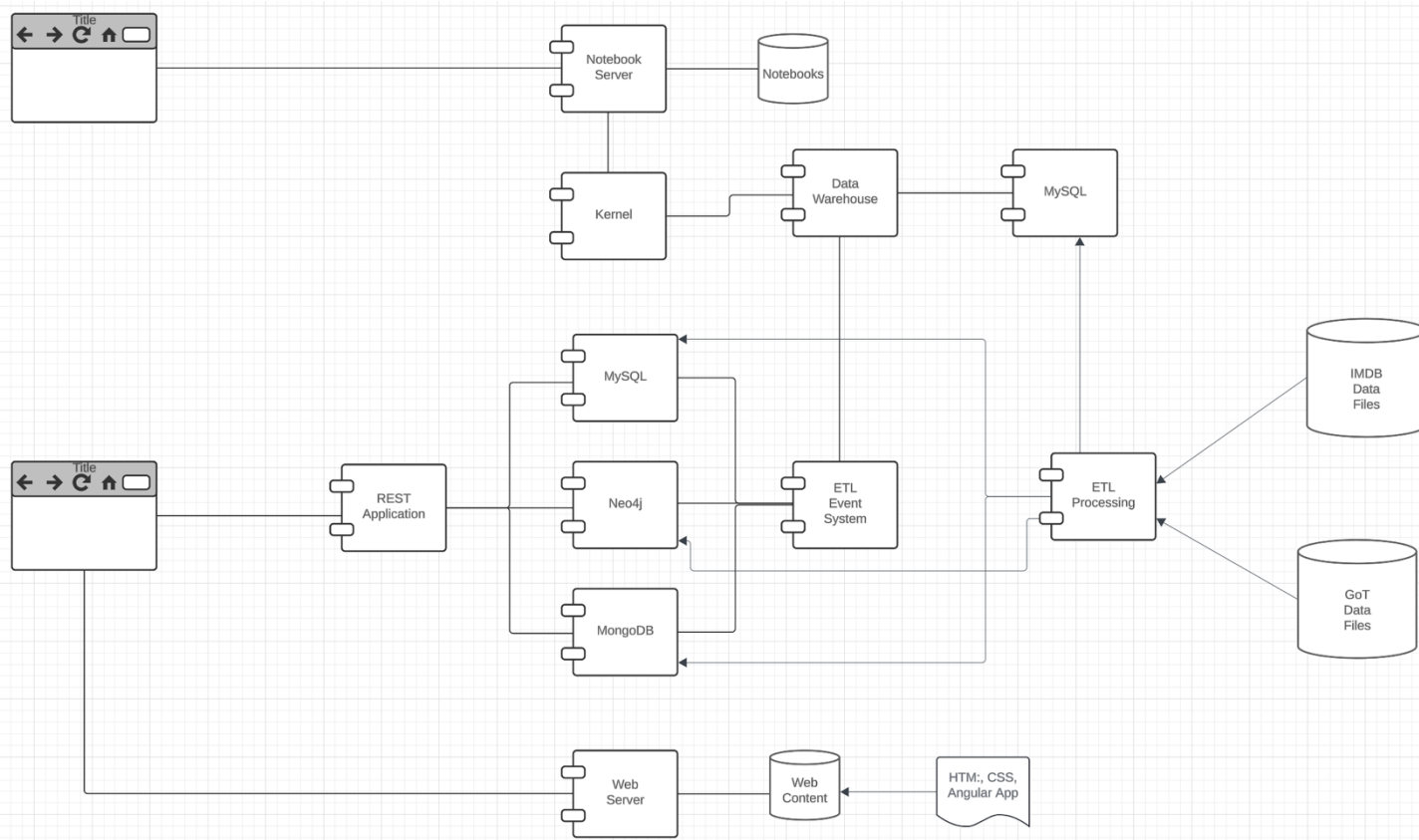


Update on Project

Vision



Vision



MongoDB Continued